



Contenido

UNIDAD 8 Bloque III. Desarrollo de aplicaciones en Laravel	2
Bases de Datos	2
Configuración inicial	2
Configuración de la Base de Datos	2
Crear la base de datos	3
Migraciones	4
Crear una nueva migración	4
Estructura de una migración	5
Ejecutar migraciones	5
SchemaBuilder	6
Crear y borrar una tabla	6
Añadir columnas	7
Añadir índices	8
Claves ajenas	8
Inicialización de la base de datos (DatabaseSeeding)	9
Crear ficheros semilla	10
Ejecutar la inicialización de datos	10
Constructor de consultas (QueryBuilder)	11
Consultas	11
Modelos de datos mediante ORM	13



UNIDAD 8 Bloque III. Desarrollo de aplicaciones en Laravel

Bases de Datos

Laravel facilita la configuración y el uso de diferentes tipos de base de datos: MySQL, Postgres, SQLite y SQL Server. En el fichero de configuración (`config/database.php`) tenemos que indicar todos los parámetros de acceso a nuestras bases de datos y además especificar cuál es la conexión que se utilizará por defecto. En Laravel podemos hacer uso de varias bases de datos a la vez, aunque sean de distinto tipo. Por defecto se accederá a la que especifiquemos en la configuración y si queremos acceder a otra conexión lo tendremos que indicar expresamente al realizar la consulta.

En este capítulo veremos cómo configurar una base de datos, como crear tablas y especificar sus campos desde código, como inicializar la base de datos y como construir consultas tanto de forma directa como a través del ORM llamado *Eloquent*.

Configuración inicial

En este primer apartado vamos a ver los primeros pasos que tenemos que dar con Laravel para empezar a trabajar con bases de datos. Para esto vamos a ver a continuación como definir la configuración de acceso, como crear una base de datos y como crear la tabla de migraciones, necesaria para crear el resto de tablas.

Configuración de la Base de Datos

Lo primero que tenemos que hacer para trabajar con bases de datos es completar la configuración. Como ejemplo vamos a configurar el acceso a una base de datos tipo MySQL. Si editamos el fichero con la configuración `config/database.php` podemos ver en primer lugar la siguiente línea:

```
'default' =>env('DB_CONNECTION', 'mysql'),
```

Este valor indica el tipo de base de datos a utilizar por defecto. Como vimos en el primer capítulo Laravel utiliza el sistema de variables de entorno para separar las distintas configuraciones de usuario o de máquina. El método `env('DB_CONNECTION', 'mysql')` lo que hace es obtener el valor de la variable `DB_CONNECTION` del fichero `.env`. En caso de que dicha variable no esté definida devolverá el valor por defecto `mysql`.

En este mismo fichero de configuración, dentro de la sección `connections`, podemos encontrar todos los campos utilizados para configurar cada tipo de base de datos, en concreto la base de datos tipo `mysql` tiene los siguientes valores:

```
'mysql' => [  
    'driver' =>'mysql',  
    'host' =>env('DB_HOST', 'localhost'),  
    'database' =>env('DB_DATABASE', 'forge'), // Nombre de la base de datos
```



```
'username' =>env('DB_USERNAME', 'forge'), // Usuario de acceso a labd  
'password' =>env('DB_PASSWORD', ''), // Contraseña de acceso  
'charset' =>'utf8',  
'collation' =>'utf8_unicode_ci','prefix' =>'',  
'strict' =>false,  
,
```

Como se puede ver, básicamente los campos que tenemos que configurar para usar nuestra base de datos son: *host*, *database*, *username* y *password*. El *host* lo podemos dejar como está si vamos a usar una base de datos local, mientras que los otros tres campos sí que tenemos que actualizarlos con los nombres de la base de datos a utilizar, el usuario y la contraseña de acceso. Para poner estos valores abrimos el fichero `.env` de la raíz del proyecto y los actualizamos:

```
DB_CONNECTION=mysql  
DB_HOST=localhost  
DB_DATABASE=nombre-base-de-datos  
DB_USERNAME=nombre-de-usuario  
DB_PASSWORD=contraseña-de-acceso
```

Crear la base de datos

Para crear la base de datos que vamos a utilizar en MySQL podemos utilizar la herramienta *PHPMyAdmin* que se ha instalado con el paquete XAMPP. Para esto accedemos a la ruta:

```
http://localhost/phpmyadmin
```

La cual nos mostrará un panel para la gestión de las bases de datos de MySQL, que nos permite, además de realizar cualquier tipo de consulta SQL, crear nuevas bases de datos o tablas, e insertar, modificar o eliminar los datos directamente. En nuestro caso apretamos en la pestaña "Bases de datos" y creamos una nueva base de datos. El nombre que le pongamos tiene que ser el mismo que el que hayamos indicado en el fichero de configuración de Laravel.

A continuación vamos a crear la tabla de migraciones. Laravel utiliza las migraciones para poder definir y crear las tablas de la base de datos desde código, y de esta manera tener un control de las versiones de las mismas.

Para poder empezar a trabajar con las migraciones es necesario en primer lugar crear la tabla de migraciones. Para esto tenemos que ejecutar el siguiente comando de Artisan:

```
php artisan migrate:install
```

Nota: Si nos diese algún error tendremos que revisar la configuración que hemos puesto de la base de datos y si hemos creado la base de datos con el nombre, usuario y contraseña indicado.



Si todo funciona correctamente ahora podemos ir al navegador y acceder de nuevo a nuestra base de datos con PHPMyAdmin, podremos ver que se nos habrá creado la tabla `migrations`. Con esto ya tenemos configurada la base de datos y el acceso a la misma. En las siguientes secciones veremos cómo añadir tablas y posteriormente como realizar consultas.

Migraciones

Las migraciones son un sistema de control de versiones para bases de datos. Permiten que un equipo trabaje sobre una base de datos añadiendo y modificando campos, manteniendo un histórico de los cambios realizados y del estado actual de la base de datos. Las migraciones se utilizan de forma conjunta con la herramienta *Schemabuilder* (que veremos en la siguiente sección) para gestionar el esquema de base de datos de la aplicación.

La forma de funcionar de las migraciones es crear ficheros (PHP) con la descripción de la tabla a crear y posteriormente, si se quiere modificar dicha tabla se añadiría una nueva migración (un nuevo fichero PHP) con los campos a modificar. Artisan incluye comandos para crear migraciones, para ejecutar las migraciones o para hacer *rollback* de las mismas (volver atrás).

Crear una nueva migración

Para crear una nueva migración se utiliza el comando de Artisan `make:migration`, al cual le pasaremos el nombre del fichero a crear y el nombre de la tabla:

```
php artisan make:migration create_users_table --create=users
```

Esto nos creará un fichero de migración en la carpeta `database/migrations` con el nombre `<TIMESTAMP>_create_users_table.php`.

Al añadir un *timestamp* a las migraciones el sistema sabe el orden en el que tiene que ejecutar (o deshacer) las mismas.

Si lo que queremos es añadir una migración que modifique los campos de una tabla existente tendremos que ejecutar el siguiente comando:

```
php artisan make:migration add_votes_to_user_table --table=users
```

En este caso se creará también un fichero en la misma carpeta, con el nombre `<TIMESTAMP>_add_votes_to_user_table.php` pero preparado para modificar los campos de dicha tabla.

Por defecto, al indicar el nombre del fichero de migraciones se suele seguir siempre el mismo patrón (aunque en realidad el nombre es libre). Si es una migración que crea una tabla el nombre tendrá que ser `create_<table-name>_table` y si es una migración que modifica una tabla será `<action>_to_<table-name>_table`



Estructura de una migración

El fichero o clase PHP generada para una migración siempre tiene una estructura similar a la siguiente:

```
<?php use Illuminate\Database\Schema\Blueprint;
use Illuminate\Database\Migrations\Migration;
```

```
class CreateUsersTable extends Migration
{
    /**
     * Run the migrations.
     * @return void
     */
    public function up()
    {
        //
    }

    /**
     * Reverse the migrations.
     * @return void
     */
    public function down()
    {
        //
    }
}
```

En el método `up` es donde tendremos crear o modificar la tabla, y en el método `down` tendremos que deshacer los cambios que se hagan en el `up` (eliminar la tabla o eliminar el campo que se haya añadido). Esto nos permitirá poder ir añadiendo y eliminando cambios sobre la base de datos y tener un control o histórico de los mismos.

Ejecutar migraciones

Después de crear una migración y de definir los campos de la tabla (en la siguiente sección veremos cómo especificar esto) tenemos que lanzar la migración con el siguiente comando:

```
php artisan migrate
```

Nota: Si nos aparece el error "class not found" lo podremos solucionar llamando a `composer dump -autoload` y volviendo a lanzar las migraciones.

Este comando aplicará la migración sobre la base de datos. Si hubiera más de una migración pendiente se ejecutarán todas. Para cada migración se llamará a su método `up` para que cree o modifique la base de datos. Posteriormente en caso de que queramos deshacer los últimos cambios podremos ejecutar:

```
php artisan migrate:rollback
```



```
# O si queremos deshacer todas las migraciones  
php artisan migrate:refresh
```

Un comando interesante cuando estamos desarrollando un nuevo sitio web es `migrate:refresh`, el cual desharrá todos los cambios y volver a aplicar las migraciones:

```
php artisan migrate:refresh
```

Además si queremos comprobar el estado de las migraciones, para ver las que ya están instaladas y las que quedan pendientes, podemos ejecutar:

```
php artisan migrate:status
```

SchemaBuilder

Una vez creada una migración tenemos que completar sus métodos `up` y `down` para indicar la tabla que queremos crear o el campo que queremos modificar. En el método `down` siempre tendremos que añadir la operación inversa, eliminar la tabla que se ha creado en el método `up` o eliminar la columna que se ha añadido. Esto nos permitirá deshacer migraciones dejando la base de datos en el mismo estado en el que se encontraban antes de que se añadieran.

Para especificar la tabla a crear o modificar, así como las columnas y tipos de datos de las mismas, se utiliza la clase *Schema*. Esta clase tiene una serie de métodos que nos permitirá especificar la estructura de las tablas independientemente del sistema de base de datos que utilicemos.

Crear y borrar una tabla

Para añadir una nueva tabla a la base de datos se utiliza el siguiente constructor:

```
Schema::create('users', function(Blueprint $table){  
    $table->increments('id');  
});
```

Donde el primer argumento es el nombre de la tabla y el segundo es una función que recibe como parámetro un objeto del tipo *Blueprint* que utilizaremos para configurar las columnas de la tabla.

En la sección `down` de la migración tendremos que eliminar la tabla que hemos creado, para esto usaremos alguno de los siguientes métodos:

```
Schema::drop('users');  
Schema::dropIfExists('users');
```

Al crear una migración con el comando de Artisan `make:migration` ya nos viene este código añadido por defecto, la creación y eliminación de la tabla que se ha indicado y

además se añaden un par de columnas por defecto (*id* y *timestamps*).

Añadir columnas

El constructor `Schema::create` recibe como segundo parámetro una función que nos permite especificar las columnas que va a tener dicha tabla. En esta función podemos ir añadiendo todos los campos que queramos, indicando para cada uno de ellos su tipo y nombre, y además si queremos también podremos indicar una serie de modificadores como valor por defecto, índices, etc. Por ejemplo:

Schema define muchos tipos de datos que podemos utilizar para definir las columnas de una tabla, algunos de los principales son:

Comando	Tipo de campo
<code>\$table->boolean('confirmed');</code>	BOOLEAN
<code>\$table->enum('choices', array('foo', 'bar'));</code>	ENUM
<code>\$table->float('amount');</code>	FLOAT
<code>\$table->increments('id');</code>	Clave principal tipo INTEGER con Auto-Increment
<code>\$table->integer('votes');</code>	INTEGER
<code>\$table->mediumInteger('numbers');</code>	MEDIUMINT
<code>\$table->smallInteger('votes');</code>	SMALLINT
<code>\$table->tinyInteger('numbers');</code>	TINYINT
<code>\$table->string('email');</code>	VARCHAR
<code>\$table->string('name', 100);</code>	VARCHAR con la longitud indicada
<code>\$table->text('description');</code>	TEXT
<code>\$table->timestamp('added_on');</code>	TIMESTAMP
<code>\$table->timestamps();</code>	Añade los <i>timestamps</i> "created_at" y "updated_at"
<code>->nullable()</code>	Indicar que la columna permite valores NULL
<code>->default(\$value)</code>	Declare a default value for a column
<code>->unsigned()</code>	Añade UNSIGNED a las columnas tipo INTEGER

Los tres últimos se pueden combinar con el resto de tipos para crear, por ejemplo, una columna que permita nulos, con un valor por defecto y de tipo *unsigned*.

Para consultar todos los tipos de datos que podemos utilizar podéis consultar la documentación de Laravel en:

<http://laravel.com/docs/5.1/migrations#creating-columns>



Añadir índices

Schema soporta los siguientes tipos de índices:

Comando	Descripción
<code>\$table->primary('id');</code>	Añadir una clave primaria
<code>\$table->primary(array('first', 'last'));</code>	Definir una clave primaria compuesta
<code>\$table->unique('email');</code>	Definir el campo como UNIQUE
<code>\$table->index('state');</code>	Añadir un índice a una columna

En la tabla se especifica como añadir estos índices después de crear el campo, pero también permite indicar estos índices a la vez que se crea el campo:

```
$table->string('email')->unique();
```

Claves ajenas

Con *Schema* también podemos definir claves ajenas entre tablas:

```
$table->integer('user_id')->unsigned();  
$table->foreign('user_id')->references('id')->on('users');
```

En este ejemplo en primer lugar añadimos la columna "user_id" de tipo UNSIGNED INTEGER (siempre tendremos que crear primero la columna sobre la que se va a aplicar la clave ajena). A continuación creamos la clave ajena entre la columna "user_id" y la columna "id" de la tabla "users".

Nota: La columna con la clave ajena tiene que ser **del mismo tipo** que la columna a la que apunta. Si por ejemplo creamos una columna a un índice auto-incremental tendremos que especificar que la columna sea *unsigned* para que no se produzcan errores.

También podemos especificar las acciones que se tienen que realizar para "onDelete" y "onupdate":

```
$table->foreign('user_id')  
->references('id')->on('users')  
->onDelete('cascade');
```

Para eliminar una clave ajena, en el método `down` de la migración tenemos que utilizar el siguiente código:

```
$table->dropForeign('posts_user_id_foreign');
```

Para indicar la clave ajena a eliminar tenemos que seguir el siguiente patrón para especificar el nombre `<tabla>_<columna>_foreign`.

Donde "tabla" es el nombre de la tabla actual y "columna" el nombre de la columna sobre la que se creó la clave ajena.



Inicialización de la base de datos (DatabaseSeeding)

Laravel también facilita la inserción de datos iniciales o datos *semilla* en la base de datos. Esta opción es muy útil para tener datos de prueba cuando estamos desarrollando una web o para crear tablas que ya tienen que contener una serie de datos en producción.

Los ficheros de "semillas" se encuentran en la carpeta `database/seeds`. Por defecto Laravel incluye el fichero `DatabaseSeeder` con el siguiente contenido:

```
<?php use Illuminate\Database\Seeder;
use Illuminate\Database\Eloquent\Model;

class DatabaseSeeder extends Seeder
{
    /**
     * Run the database seeds.
     * @return void
     */ public function run()
    {
        //...
    }
}
```

Al lanzar la inicialización se llamará por defecto al método `run` de la clase `DatabaseSeeder`. Desde aquí podemos crear las semillas de varias formas:

- Escribir el código para insertar los datos dentro del propio método `run`
- Crear otros métodos dentro de la clase `DatabaseSeeder` y llamarlos desde el método `run`. De esta forma podemos separar mejor las inicializaciones.
- Crear otros ficheros *Seeder* y llamarlos desde el método `run` es la clase principal.

Según lo que vayamos a hacer nos puede interesar una opción u otra. Por ejemplo, si el código que vamos a escribir es poco nos puede sobrar con las opciones 1 o 2, sin embargo si vamos a trabajar bastante con las inicializaciones quizás lo mejor es la opción 3.

A continuación se incluye un ejemplo de la opción 1:

```
Class DatabaseSeeder extends Seeder
{
    Public function run()
    {
        // Borramos los datos de la tabla
        DB::table('users')->delete();

        // Añadimos una entrada a esta tabla
        User::create(array('email' => 'foo@bar.com'));
    }
}
```



Como se puede ver en el ejemplo en general tendremos que eliminar primero los datos de la tabla en cuestión y posteriormente añadir los datos. Para insertar datos en una tabla podemos utilizar el método que se usa en el ejemplo o alguna de las otras opciones que se verán en las siguientes secciones sobre "Constructor de consultas" y "Eloquent ORM".

Crear ficheros semilla

Como hemos visto en el apartado anterior, podemos crear más ficheros o clases *semilla* para modularizar mejor el código de las inicializaciones. De esta forma podemos crear un fichero de semillas para cada una de las tablas o modelos de datos que tengamos.

En la carpeta `database/seeds` podemos añadir más ficheros PHP con clases que extiendan de `Seeder` para definir nuestros propios ficheros de "semillas". El nombre de los ficheros suele seguir el mismo patrón `<nombre-tabla>TableSeeder`, por ejemplo `"UsersTableSeeder"`. Artisan incluye un comando que nos facilitará crear los ficheros de semillas y que además incluirán las estructuras base de la clase. Por ejemplo, para crear el fichero de inicialización de la tabla de usuarios haríamos:

```
php artisan make:seeder UsersTableSeeder
```

Para que esta nueva clase se ejecute tenemos que llamarla desde el método `run` de la clase principal `DatabaseSeeder` de la forma:

```
Class Database Seeder extends Seeder
{
    Public function run()
    {
        Model::unguard();
        $this->call(UsersTableSeeder::class);
        Model::reguard();
    }
}
```

El método `call` lo que hace es llamar al método `run` de la clase indicada. Además en el ejemplo hemos añadido las llamadas a `unguard` y a `reguard`, que lo que hacen es desactivar y volver a activar (respectivamente) la inserción de datos masiva o por lotes.

Ejecutar la inicialización de datos

Una vez definidos los ficheros de semillas, cuando queramos ejecutarlos para rellenar de datos la base de datos tendremos que usar el siguiente comando de Artisan:

```
php artisan db:seed
```



Constructor de consultas (QueryBuilder)

Laravel incluye una serie de clases que nos facilita la construcción de consultas y otro tipo de operaciones con la base de datos. Además, al utilizar estas clases, creamos una notación mucho más legible, compatible con todos los tipos de bases de datos soportados por Laravel y que nos previene de cometer errores o de ataques por inyección de código SQL.

Consultas

Para realizar una "Select" que devuelva todas las filas de una tabla utilizaremos el siguiente código:

```
$users = DB::table('users')->get();  
foreach ($users as $user)  
{  
    echo $user->name;  
}
```

En el ejemplo se utiliza el constructor `DB::table()` indicando el nombre de la tabla sobre la que se va a realizar la consulta, y por último se llama al método `get()` para obtener todas las filas de la misma.

Si queremos obtener un solo elemento podemos utilizar `first()` en lugar de `get()`, de la forma:

```
$user = DB::table('users')->first();  
echo $user->name;
```

Clausula where

Para filtrar los datos usamos la clausula `where`, indicando el nombre de la columna y el valor a filtrar:

```
$user = DB::table('users')->where('name', 'Pedro')->get();  
echo $user->name;
```

En este ejemplo, la clausula `where` filtrará todas las filas cuya columna `name` sea igual a Pedro. Si queremos realizar otro tipo de filtrados, como columnas que tengan un valor mayor (`>`), mayor o igual (`>=`), menor (`<`), menor o igual (`<=`), distinto del indicado (`<>`) o usar el operador `like`, lo podemos indicar como segundo parámetro de la forma:

```
$users = DB::table('users')->where('votes', '>', 100)->get();  
  
$users = DB::table('users')->where('status', '<>', 'active')->get();  
  
$users = DB::table('users')->where('name', 'like', 'T%')->get();
```

Si añadimos más clausulas `where` a la consulta por defecto se unirán mediante el operador lógico AND. En caso de que queramos utilizar el operador lógico OR lo tendremos que



realizar usando `orWhere` de la forma:

```
$users = DB::table('users')
->where('votes', '>', 100)
->orWhere('name', 'Pedro')
->get();
```

orderBy / groupBy / having_

También podemos utilizar los métodos `orderBy`, `groupBy` y `having` en las consultas:

```
$users = DB::table('users')
->orderBy('name', 'desc')
->groupBy('count')
->having('count', '>', 100)
->get();
```

Offset / Limit

Si queremos indicar un *offset* o *limit* lo realizaremos mediante los métodos `skip`

(para el *offset*) y `take` (para *limit*), por ejemplo:

```
$users = DB::table('users')->skip(10)->take(5)->get();
```

Transacciones

Laravel también permite crear transacciones sobre un conjunto de operaciones:

```
DB::transaction(function()
{
DB::table('users')->update(array('votes' =>1));

DB::table('posts')->delete();
});
```

En caso de que se produzca cualquier excepción en las operaciones que se realizan en la transacción se desharian todos los cambios aplicados hasta ese momento de forma automática.

Más información

Para más información sobre la construcción de *Querys* (*join*, *insert*, *update*, *delete*, agregados, etc.) podéis consultar la documentación de Laravel en [sú sitio web](https://laravel.com/docs):

<http://laravel.com/docs/5.1/queries>

Modelos de datos mediante ORM

El mapeado objeto-relacional (más conocido por su nombre en inglés, *Object-Relationalmapping*, o por sus siglas ORM) es una técnica de programación para convertir datos entre un lenguaje de programación orientado a objetos y una base de datos relacional como motor de persistencia. Esto posibilita el uso de las características propias de la orientación a objetos, podremos acceder directamente a los campos de un objeto para leer los datos de una base de datos o para insertarlos o modificarlos.

Laravel incluye su propio sistema de ORM llamado *Eloquent*, el cual nos proporciona una manera elegante y fácil de interactuar con la base de datos. Para cada tabla de la base de datos tendremos que definir su correspondiente modelo, el cual se utilizará para interactuar desde código con la tabla.

Definición de un modelo

Por defecto los modelos se guardarán como clases PHP dentro de la carpeta `app`, sin embargo Laravel nos da libertad para colocarlos en otra carpeta si queremos, como por ejemplo la carpeta `app/Models`. Pero en este caso tendremos que asegurarnos de indicar correctamente el espacio de nombres.

Para definir un modelo que use *Eloquent* únicamente tenemos que crear una clase que herede de la clase `Model`:

```
<?php namespace App;
use Illuminate\Database\Eloquent\Model; class User extends Model
{
//...
}
```

Sin embargo es mucho más fácil y rápido crear los modelos usando el comando `make:model` de Artisan:

```
php artisan make:model User
```

Este comando creará el fichero `User.php` dentro de la carpeta `app` con el código básico de un modelo que hemos visto en el ejemplo anterior.

Convenios en EloquentNombre

En general el nombre de los modelos se pone en singular con la primera letra en mayúscula, mientras que el nombre de las tablas suele estar en plural.

Gracias a esto, al definir un modelo no es necesario indicar el nombre de la tabla asociada, sino que *Eloquent* automáticamente buscará la tabla transformando el nombre del modelo a minúsculas y buscando su plural (en inglés). En el ejemplo anterior que hemos



creado el modelo `User` buscará la tabla de la base de datos llamada `users` y en caso de no encontrarla daría un error.

Si la tabla tuviese otro nombre lo podemos indicar usando la propiedad protegida `$table` del modelo:

```
<?php namespace App;
use Illuminate\Database\Eloquent\Model; class User extends Model
{
protected $table = 'my_users';
}
```

Clave primaria

Laravel también asume que cada tabla tiene declarada una clave primaria con el nombre `id`. En el caso de que no sea así y queramos cambiarlo tendremos que sobrescribir el valor de la propiedad protegida `$primaryKey` del modelo, por ejemplo:

```
protected $primaryKey = 'my_id';
```

Nota: Es importante definir correctamente este valor ya que se utiliza en determinados métodos de *Eloquent*, como por ejemplo para buscar registros o para crear las relaciones entre modelos.

Timestamps

Otra propiedad que en ocasiones tendremos que establecer son los *timestamps* automáticos. Por defecto *Eloquent* asume que todas las tablas contienen los campos `updated_at` y `created_at` (los cuales los podemos añadir muy fácilmente con *Schema* añadiendo `$table->timestamps()` en la migración). Estos campos se actualizarán automáticamente cuando se cree un nuevo registro o se modifique. En el caso de que no queramos utilizarlos (y que no estén añadidos a la tabla) tendremos que indicarlo en el modelo o de otra forma nos daría un error. Para indicar que no los actualice automáticamente tendremos que modificar el valor de la propiedad pública `$timestamps` a *false*, por ejemplo:

```
public $timestamps = false;
```

A continuación se muestra un ejemplo de un modelo de *Eloquent* en el que se añaden todas las especificaciones que hemos visto:

```
<?php namespace App;
use Illuminate\Database\Eloquent\Model;
class User extends Model
{
protected $table = 'my_users';
protected $primaryKey = 'my_id' public $timestamps = false;
}
```



Uso de un modelo de datos

Una vez creado el modelo ya podemos empezar a utilizarlo para recuperar datos de la base de datos, para insertar nuevos datos o para actualizarlos. El sitio correcto donde realizar estas acciones es en el controlador, el cual se los tendrá que pasar a la vista ya preparados para su visualización.

Es importante que para su utilización indiquemos al inicio de la clase el espacio de nombres del modelo o modelos a utilizar. Por ejemplo, si vamos a usar los modelos `User` y `Orders` tendríamos que añadir:

```
use App\User;
use App\Orders;
```

Consultar datos

Para obtener todas las filas de la tabla asociada a un modelo usaremos el método `all()`:

```
$users = User::all();
foreach($users as $user) {
    echo $user->name;
}
```

Este método nos devolverá un array de resultados, donde cada item del array será una instancia del modelo `User`. Gracias a esto al obtener un elemento del array podemos acceder a los campos o columnas de la tabla como si fueran propiedades del objeto (`$user->name`).

Nota: Todos los métodos que se describen en la sección de "Constructor de consultas" y en la documentación de Laravel sobre "QueryBuilder" también se pueden utilizar en los modelos Eloquent. Por lo tanto podremos utilizar *where*, *orWhere*, *first*, *get*, *orderBy*, *groupBy*, *having*, *skip*, *take*, etc. para elaborar las consultas.

Eloquent también incorpora el método `find($id)` para buscar un elemento a partir del identificador único del modelo, por ejemplo:

```
$user = User::find(1);
echo $user->name;
```

Si queremos que se lance una excepción cuando no se encuentre un modelo podemos utilizar los métodos `findOrFail` o `firstOrFail`. Esto nos permite capturar las excepciones y mostrar un error 404 cuando sucedan.

```
$model = User::findOrFail(1);
$model = User::where('votes', '>', 100)->firstOrFail();
```

A continuación se incluyen otros ejemplos de consultas usando Eloquent con algunos de



los métodos que ya habíamos visto en la sección "Constructor de consultas":

```
// Obtener 10 usuarios con más de 100 votos
$users = User::where('votes', '>', 100)->take(10)->get();

// Obtener el primer usuario con más de 100 votos
$user = User::where('votes', '>', 100)->first();
```

También podemos utilizar los métodos agregados para calcular el total de registros obtenidos, o el máximo, mínimo, media o suma de una determinada columna. Por ejemplo:

```
$count = User::where('votes', '>', 100)->count();
$price = Orders::max('price');
$price = Orders::min('price');
$price = Orders::avg('price');
$total = User::sum('votes');
```

Insertar datos

Para añadir una entrada en la tabla de la base de datos asociada con un modelo simplemente tenemos que crear una nueva instancia de dicho modelo, asignar los valores que queramos y por último guardarlos con el método `save()`:

```
$user = new User;
$user->name = 'Juan';
$user->save();
```

Para obtener el identificador asignado en la base de datos después de guardar (cuando se trate de tablas con índice auto-incremental), lo podremos recuperar simplemente accediendo al campo `id` del objeto que habíamos creado, por ejemplo:

```
$insertedId = $user->id;
```

Actualizar datos

Para actualizar una instancia de un modelo es muy sencillo, solo tendremos que recuperar en primer lugar la instancia que queremos actualizar, a continuación modificarla y por último guardar los datos:

```
$user = User::find(1);
$user->email = 'juan@gmail.com';
$user->save();
```

Borrar datos

Para borrar una instancia de un modelo en la base de datos simplemente tenemos que usar su método `delete()`:

```
$user = User::find(1);
```




```
$user->delete();
```

Si por ejemplo queremos borrar un conjunto de resultados también podemos usar el método `delete()` de la forma:

```
$affectedRows = User::where('votes', '>', 100)->delete();
```

Más información

Para más información sobre cómo crear relaciones entre modelos, *eagerloading*, etc. podéis consultar directamente la documentación de Laravel en:

<http://laravel.com/docs/5.1/eloquent>